

Computational Fluid Dynamics for Aerospace Applications

Lecture 4 Partial Differential Equations

Partial Differential Equations

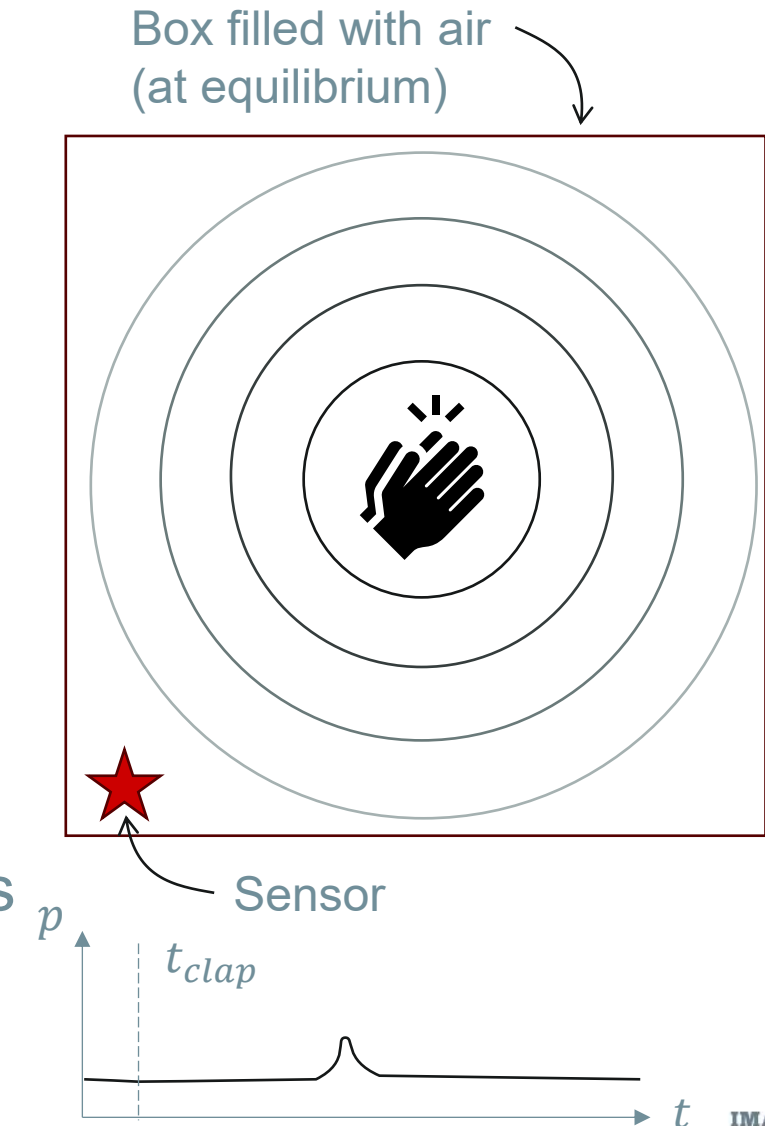
- Information Propagation
- The Grid

Information Propagation (PDEs-lite)

- We now have governing equations, what do we need to know about them to solve them?
- This part of the course usually covers PDEs
- You would usually see terms like:
 - Quasi-linear
 - Hyperbolic, parabolic, elliptic
 - Equilibrium problem, marching problem, eigenvalue problem
 - Initial value problem, boundary value problem
- We will focus on the important intuition relevant to us

Information Propagation

- What does “information” mean for fluids?
 - The ability of a point in the fluid to know about perturbations to another point in the fluid
- A “clap” emits pressure waves that may eventually reach other points
 - Those pressure waves carry information
 - Travel at the **speed of sound**
 - There is no guarantee that the pressure waves will reach every point (to be discussed)

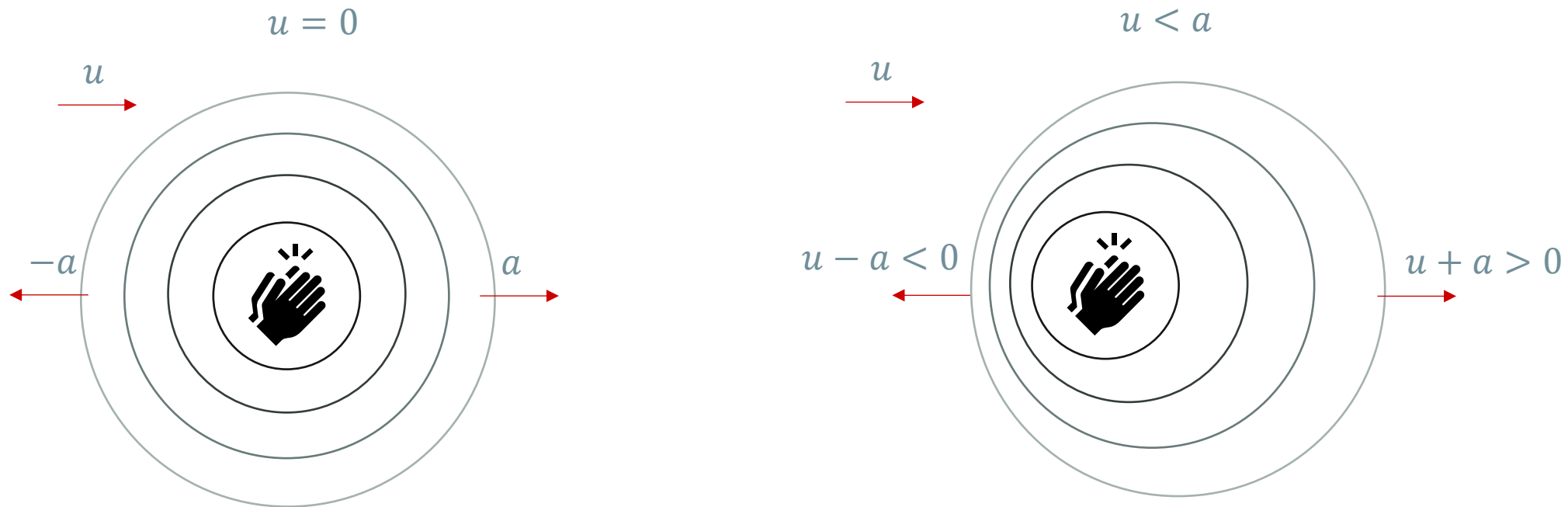


Information Propagation

- Doesn't have to be a pressure perturbation
 - Changing a boundary condition (e.g., wall temperature) would do the same thing
- Information propagation has direction
 - Spatially, it can go left, right, up, down, etc.
 - Information also propagates temporally, forward in time
- At any moment, can information have propagated further than sound waves have travelled?
 - No! At that moment, the fluid beyond the sound waves does not know that a perturbation has occurred (Domain of Independence)

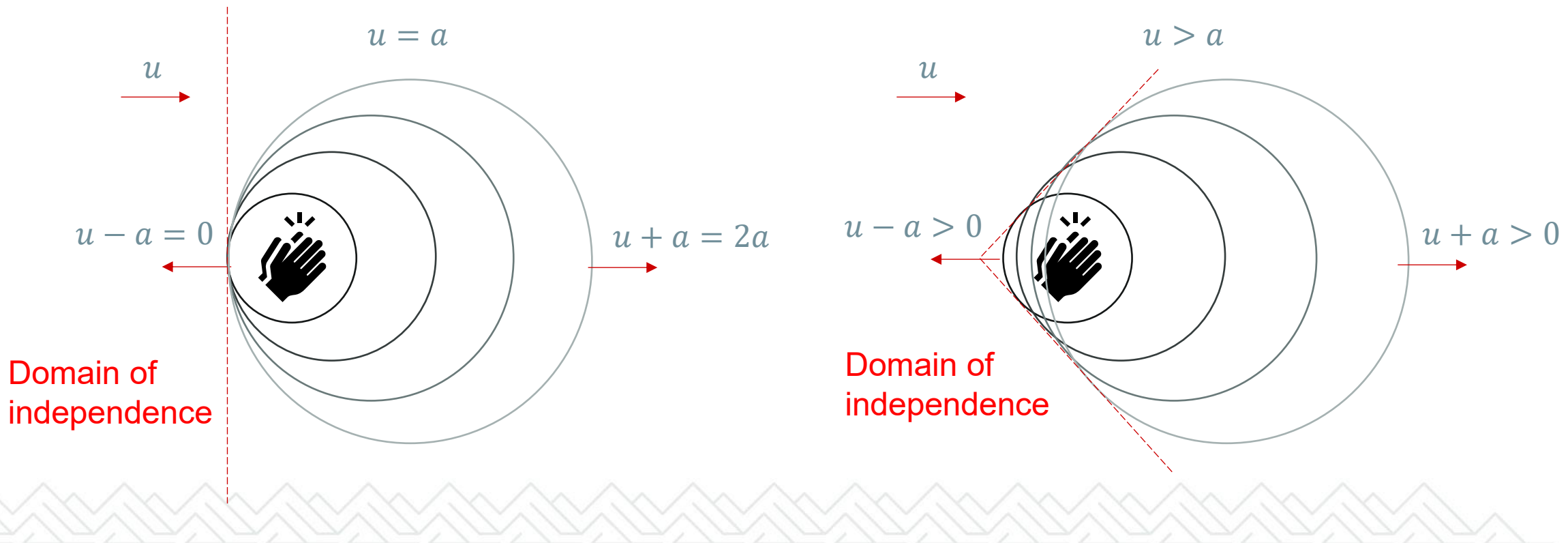
Information Propagation

- Speed of the flow adds more complications
 - Pressure waves travel at speed of sound (a)
 - Pressure waves can be advected by the flow



Information Propagation

- Speed of the flow adds more complications
 - Supersonic flows are faster than pressure waves and create regions where information will never reach



Information Propagation

- What if we're not in the real world, but considering equations with assumptions?

- Steady-state inviscid Euler

$$\rho \mathbf{V} \cdot \nabla \mathbf{V} = -\nabla p + \rho \mathbf{f}$$

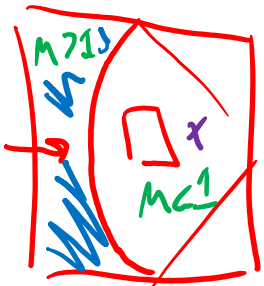
- If you remove the time derivative, you are essentially solving for the equilibrium solution and there is no concept of things moving in time
- If subsonic, adding a perturbation or changing a boundary condition immediately updates the solution everywhere – no domain of independence
- If supersonic, there are still regions where the waves will never reach, even after infinite time – yes domain of independence

- Incompressible, inviscid, Navier-Stokes

$$\nabla \cdot \mathbf{V} = 0$$

- Since such a fluid cannot compress, pressure waves don't travel at a
- This is equivalent to assuming the speed of sound is infinite
- Any perturbation is felt by every point instantly – no domain of independence

$M=3$



Domain of independence

Information Propagation

- When there are physical regions in the domain that information cannot reach, the governing PDEs are considered hyperbolic
- When information cannot travel backwards in time, the governing PDEs are considered parabolic
 - Same “Domain of Independence” idea, but temporal instead of spatial
 - This is generally associated with diffusion terms (which were purposefully not discussed on previous slides)
- When information immediately reaches every point, the governing PDEs are considered elliptic
- This classification controls how we should solve the problem, AND what kind of boundary conditions are correct

Information Propagation

Hyperbolic and parabolic systems are solved as marching problems

Elliptic systems are solved as equilibrium problems

- Compressible Navier-Stokes Equations
 - Hyperbolic and parabolic
 - This is a marching problem (i.e., we solve it one timestep at a time)
- Euler Equations (inviscid)
 - Unsteady equations are hyperbolic (marching problem)
 - Steady-state supersonic is hyperbolic (marching problem)
 - Steady-state subsonic is elliptic (equilibrium problem)
- Incompressible Navier-Stokes Equations
 - Parabolic and elliptic (is this a marching or equilibrium problem?)
 - Inviscid equations are elliptic (equilibrium problem)

Information Propagation

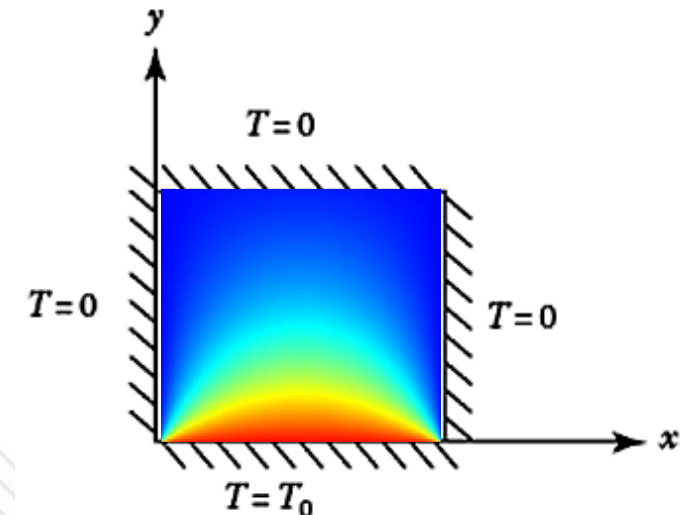
- Equilibrium problems
 - Occurs when a solution to a PDE is desired in a closed domain subject to prescribed boundary conditions
 - Essentially, a boundary value problem
- Consider finding the steady-state temperature distribution in a two-dimensional conducting medium

- Laplace's equation

- $$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

Dependent on supplied
boundary conditions

$$\begin{aligned} T(0, y) &= 0 \\ T(1, y) &= 0 \\ T(x, 0) &= T_0 \\ T(x, 1) &= 0 \end{aligned}$$



Information Propagation

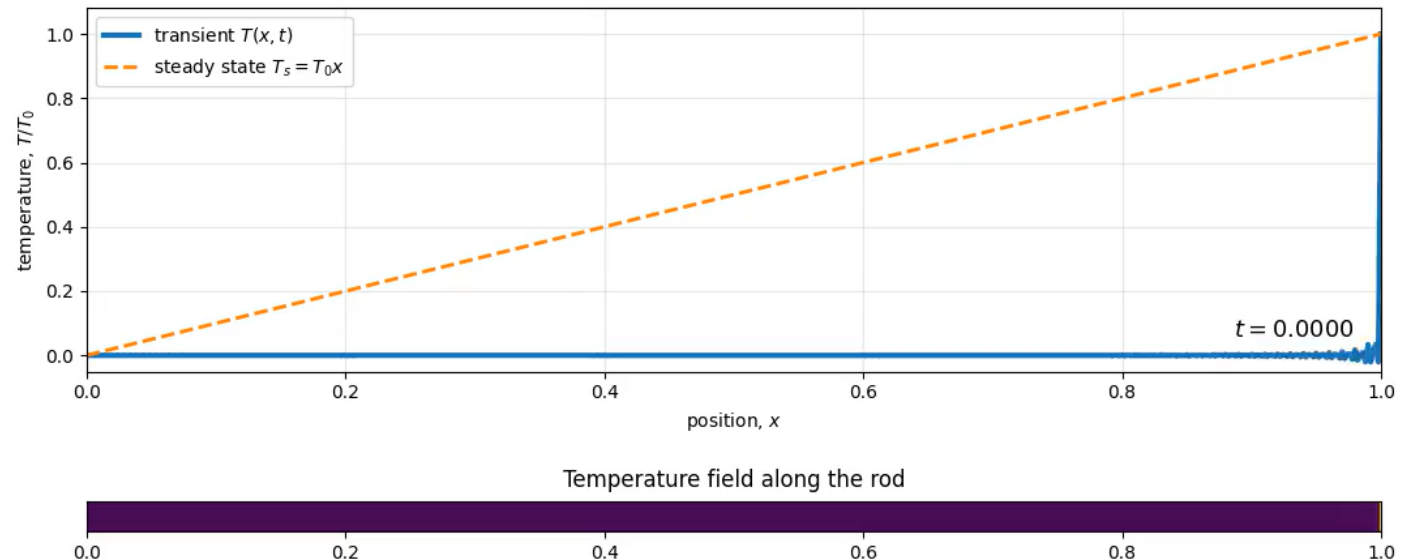
- Marching problems
 - Transient(-like) problems where the solution of a PDE is required in a domain subject to a set of initial conditions and boundary conditions
 - Essentially, an initial value or initial boundary value problem
- Consider finding the unsteady evolution of temperature in a one-dimensional conducting medium

$$\bullet \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2}$$

Boundary
conditions

$$\begin{aligned} \rightarrow T(0, t) &= 0 \\ \rightarrow T(1, t) &= T_0 \end{aligned}$$

Initial condition $\rightarrow T(x, 0) = 0$



Information Propagation

For the first half of this class,
we will focus on marching
problems exclusively

(and therefore, will not focus on solving steady equations or
incompressible Navier-Stokes equations)

The Grid

- We now know something about our PDEs, but how do we actually solve them?
- For example, how would we even approach a solution to the inviscid energy equation?

$$\frac{\partial(\rho e)}{\partial t} + \frac{\partial(\rho e u)}{\partial x} + \frac{\partial(\rho e v)}{\partial y} + \frac{\partial(\rho e w)}{\partial z} = \rho \dot{q} - p \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} \right)$$

- Note that all our PDEs describe the evolution in time and space of a continuous field (e.g., $e(x, y, z, t)$ above)
- We have never found an exact analytical solution to these continuous fields present in our equations

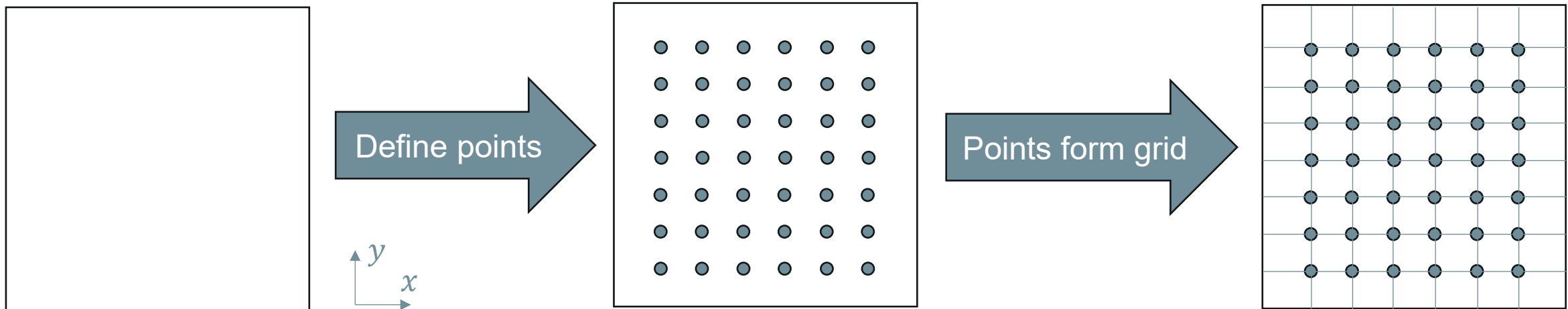
The Grid

- What if we are willing to drop the requirement that our solutions are exact and continuous?
 - A close approximation in discrete locations is still very helpful!
- Two questions
 - 1) How do we select discrete locations?
 - Today
 - 2) How do we approximate the equations?
 - Next lecture
- Start with question #1

The Grid

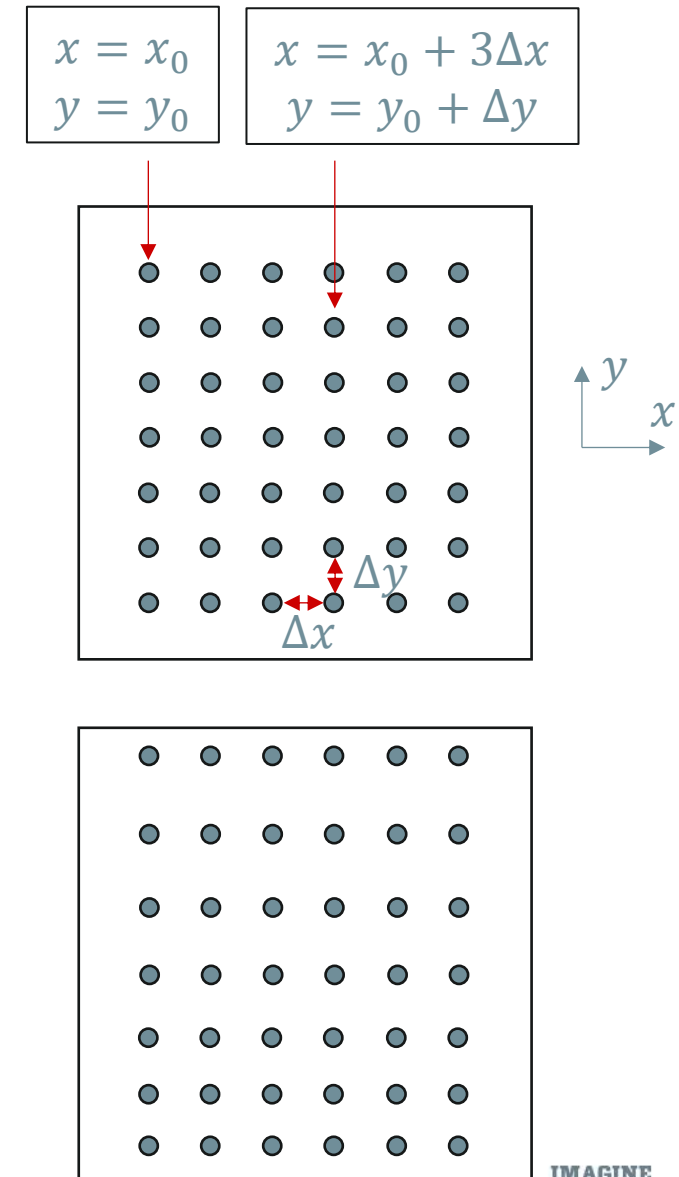
- Replace the continuous domain by a discrete mesh or grid
 - This is a collection of points that define where you will solve the PDEs

Physical domain of interest



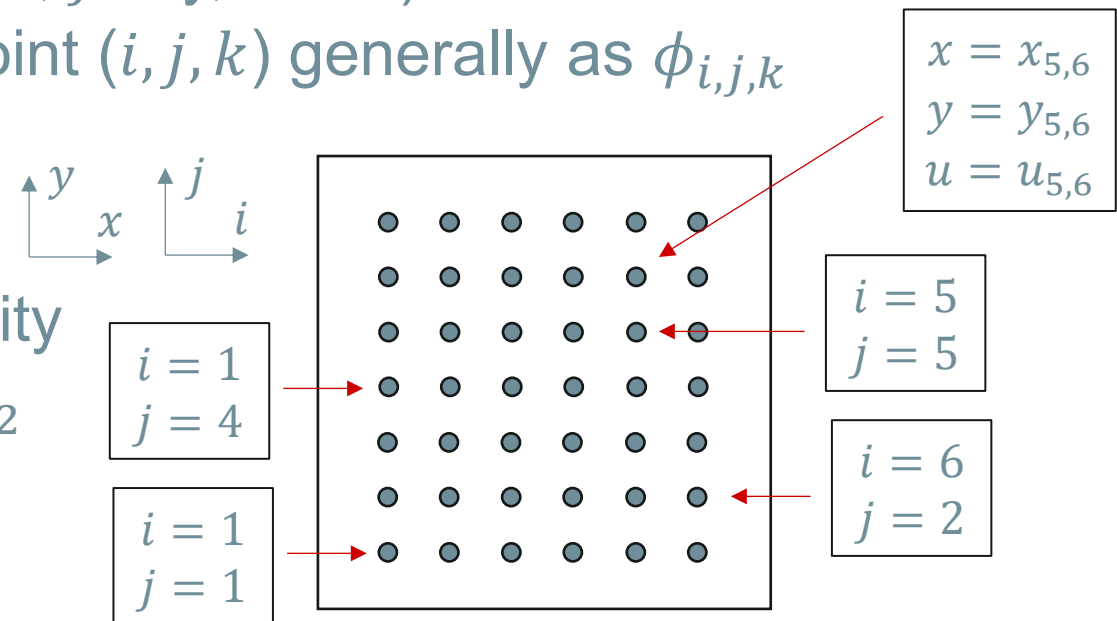
The Grid

- When we define a point, we need to define the physical coordinates of that point
- In three-dimensions, each point has a value of x, y, z
- The simplest grids have evenly spaced points
 - Δx is the x-distance between two neighboring points is a constant in this case (but doesn't have to be). Same for Δy and Δz
 - Δx does not have to equal Δy or Δz , but often will
 - Can use smaller Δ to better resolve specific regions (e.g., near the wall for a boundary layer)



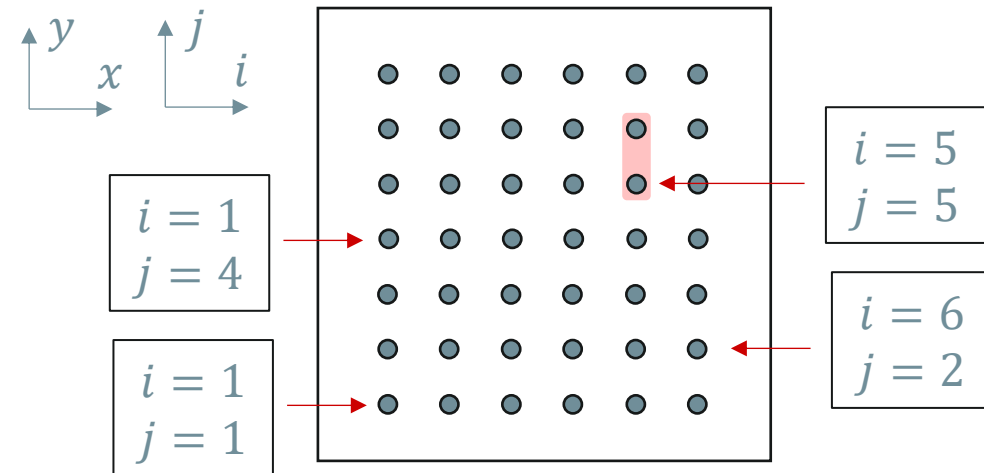
The Grid

- An important part of creating the grid is labeling points and understanding how points are connected to each other
 - Give each point a designation (i in x , j in y , k in z)
 - We label any quantity (ϕ) at any point (i, j, k) generally as $\phi_{i,j,k}$
- The physical location of the point corresponding to $i = 6, j = 2$ is given by $x_{6,2}, y_{6,2}$. The components of velocity at that point are given by $u_{6,2}, v_{6,2}, w_{6,2}$
- Note: Quantities only exist at the points. You cannot directly calculate quantities between points



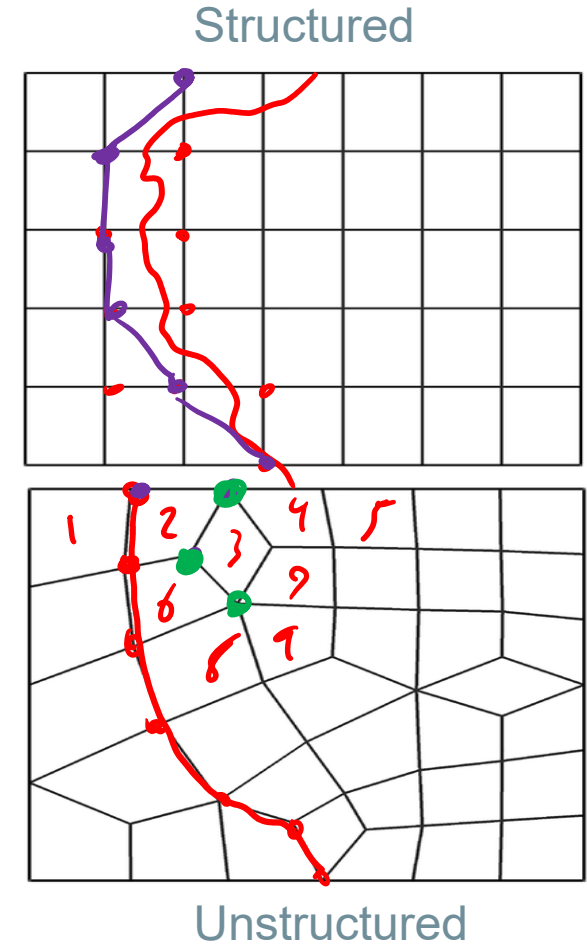
The Grid

- The benefit of defining the grid this way is that we implicitly know which points are neighbors just by i, j, k
- For example, the point given by $i = 5, j = 6$ is immediately above the point given by $i = 5, j = 5$.
- This will make calculating gradients using adjacent points much easier



The Grid

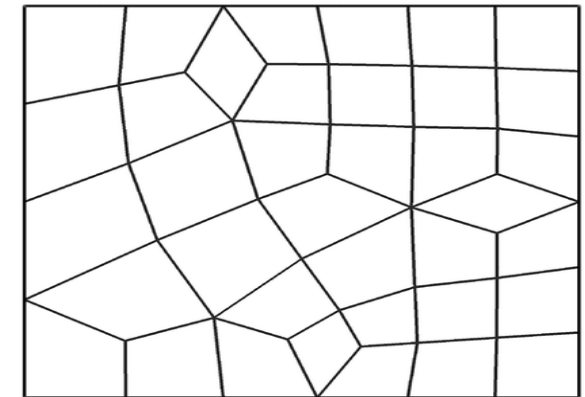
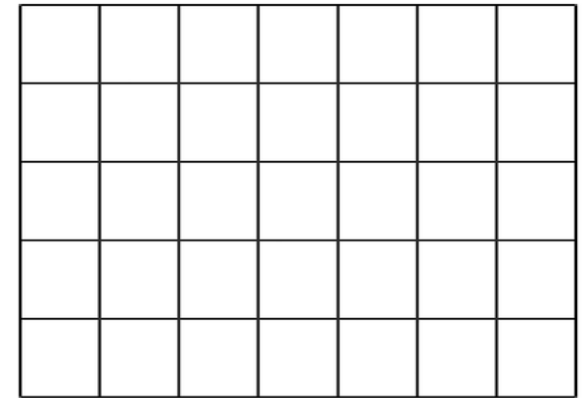
- This organized, uniform grid with a regularly defined layout (i.e., i, j, k) is a structured grid
 - Mathematically easy to work with and code, finding neighboring points is trivial
 - What if you want to simulate a curved shape?
 - Not ideal for complex geometries
- Alternatively, use an unstructured grid where points can be located anywhere
 - Points can follow complex geometries
 - Must explicitly track information about neighbors
 - Neighbors aren't simply up/down/left/right
 - Leads to more complex math and software



The Grid

- For the first half of this class, we will stick with simple structured grids
 - Since we are writing the code, keep it simple
 - Our simulations will use basic geometries
- For the second half of this class, the commercial code has both options, but we generally will use unstructured meshes
 - Someone else already developed the feature for us
 - We will use it to study more complex geometries

Structured



Unstructured

Looking Forward

- We are now ready to answer the second question from earlier
- How do we approximate the equations?
- We will take our PDEs, approximate them algebraically (while ensuring the error is at an acceptable level), and solve them on the grid we have defined